



MARYMOUNT

U N I V E R S I T Y

RECEIPT SCANNER

3/12/2026

Marymount University

Dr. Natalia Bell

IT-548-A

James Green, Chris Duckers, Numi Tesfay

This Page Intentionally Left Blank

RECEIPT SCANNER

This document provides an overview of the Receipt Scanner application software requirements, design, and usage. This document is produced for IT-548-A, Marymount University, as a group project. Volume 3 of this document is produced and stored externally.

Planning Document Completion Date:3/14/2026

Software Requirements Completion Date:

Software Design Completion Date:

Software Implementation Completion Date:

Software Usage Guide Completion Date:

ORGANIZATION

This document is organized into the following sections:

Volume-1: Receipt Scanner Planning Document

Volume-2: Receipt Scanner Software Requirements

Volume-3: Receipt Scanner Software Design Description

Volume-4: Receipt Scanner code

Volume-5: Receipt Scanner Usage Guide

This Page Intentionally Left Blank

SOFTWARE PLANNING

RECEIPT SCANNR VOLUME 1

This Page Intentionally Left Blank

CHAPTER 1

PROJECT OVERVIEW

1.1 Purpose

The purpose of this project is to explore the use of modern software development tools and artificial intelligence techniques within a Python-based application to solve a practical real-world problem. Specifically, the system aims to demonstrate how Optical Character Recognition (OCR) and automated information extraction techniques can be used to digitize paper receipts and derive meaningful financial information from them.

Many individuals and small businesses rely on paper receipts to track expenses. Manual entry of receipt data into spreadsheets or financial systems is time-consuming and prone to human error. As businesses accumulate large numbers of receipts, organizing and analyzing expense information becomes inefficient.

The Receipt Scanner application will allow a user to upload images of receipts and automatically extract key fields such as transaction date, vendor name, price paid, and gallons purchased. The extracted data will then be organized into collections of receipts defined by the user.

Using these extracted values, the system will compute summary statistics for each collection, including the average price per gallon, the total amount spent, the number of receipts in the collection, and the date range represented by the receipts.

This project is intended as a proof-of-concept system designed to demonstrate the feasibility of combining OCR technology with simple automated parsing techniques in a Python environment. The application is not intended to serve as a production-ready financial system, but rather as a prototype illustrating how automation tools can assist small single-user businesses with basic expense analysis.

1.2 Roles

The development team consists of the following members:

James Green

Responsible for system planning, requirements development, implementation of core application functionality, and documentation of system behavior.

Chris Duckers

Responsible for system design development, review of requirements and implementation artifacts, and collaboration on application development.

Numi

Responsible for creating and maintaining documentation, presentation, and users guide.

All team members will participate in system testing, review activities, and documentation refinement throughout the project lifecycle.

1.3 Deliverables

The project will produce the following deliverables:

1.3.1 Planning Document

Provides an overview of the project goals, objectives, and development strategy. This document outlines the development timeline, team responsibilities, tools and technologies used, and project assumptions. It serves as the foundation for all subsequent development activities and ensures that the team has a clear understanding of the project scope and direction

1.3.2 Software Requirements Specification (SRS)

Defines the functional and non-functional requirements of the Receipt Scanner application. This document describes the expected behavior of the system, the features it must provide, and the constraints under which it must operate. The SRS serves as a formal agreement between developers and reviewers regarding what the system must accomplish.

1.3.3 Software Design Description (SWDD)

Outlines the architectural and technical design of the Receipt Scanner system. This document explains how the system components interact, the structure of the software modules, and the technologies used to implement the application. The design document provides detailed technical information necessary for implementing and maintaining the system.

1.3.4 Application Implementation

Consists of the fully developed Receipt Scanner software written in Python. The application will include modules responsible for image upload, OCR processing, data extraction, receipt organization, and statistical analysis.

1.3.5 User Guide

Provides instructions for installing, running, and using the Receipt Scanner system. It explains how users can upload receipt images, create receipt collections, view extracted information, and analyze summary statistics.

This Page Intentionally Left Blank

CHAPTER 2

DEVELOPMENT

2.1 Assumptions

The following assumptions apply to the development and operation of the Receipt Scanner system:

- Users will provide receipt images in a clear and readable format suitable for OCR processing.
- Receipts are assumed to be standard retail receipts containing recognizable textual patterns such as vendor names, dates, and totals.
- The system will be used by a single user operating locally on their machine.
- Browser storage may impose limitations on the number of receipts that can be loaded simultaneously.
- Uploaded files will be validated to confirm they are image files before processing.
- Security concerns related to storing uploaded images locally are considered out of scope for this proof-of-concept implementation.
- Receipt collections will typically contain fewer than 100 receipts.

2.2 Tools and Languages

1. The Receipt Scanner system will be implemented entirely in Python.
2. The following technologies and tools will be used during development:
3. Flask python web framework will be used to host a local web server and provide the browser-based interface.
4. **OCR Processing**

An open-source OCR engine such as **Tesseract OCR** will be used to extract text from uploaded receipt images.

5. **Image Processing**

Python libraries such as **Pillow** or **OpenCV** may be used for image preprocessing prior to OCR extraction.

6. **Frontend Interface**

The application will provide a simple browser-based interface allowing users to upload receipts, manage receipt collections, and view extracted data summaries.

7. Receipt images will be stored on the local file system, while extracted receipt metadata will be stored within the browser session for display and analysis.
8. All tools used in this project will be open source.

2.1 Version Control

Source code and documentation for the Receipt Scanner system will be maintained using a version control system such as Git. Version control will be used to track changes to the codebase, manage revisions to project documentation, and support collaborative development between team members.

SOFTWARE REQUIREMENTS

RECEIPT SCANNER VOL 2

This Page Intentionally Left Blank

CHAPTER 3

INTRODUCTION

3.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements of the Receipt Scanner application.

The purpose of this document is to define the behavior and capabilities of the system in a clear and structured manner so that developers and reviewers can understand the expected functionality of the application prior to implementation.

3.2 Scope

The Receipt Scanner system is a Python-based application designed to digitize receipt images and extract structured information from them using Optical Character Recognition (OCR).

The system allows a user to create named collections of receipts, upload receipt images, extract relevant transaction information, and compute summary statistics based on the extracted values.

The application will operate locally and will provide a browser-based interface for interacting with the system. The system is intended as a proof-of-concept prototype and is not designed for enterprise or multi-user deployment.

3.3 System Overview

The system provides a lightweight workflow for organizing and analyzing receipt data.

Users can create collections that represent groups of receipts. Receipts may then be uploaded individually or in batches. Each uploaded image will be processed using OCR to extract textual content. The system will then attempt to identify specific receipt fields such as vendor name, transaction date, price paid, and gallons purchased.

After extraction, the system will display the extracted information and allow the user to manually edit any fields if the automatic extraction is incorrect. Receipts flagged as problematic by the extraction process will be displayed separately and excluded from summary calculations.

The system will compute several summary statistics for each collection, including:

- Average price per gallon
- Total amount spent
- Date range of receipts
- Number of valid receipts in the collection

These summaries allow users such as small single-operator businesses to quickly analyze vehicle fuel expenses.

CHAPTER 4

FUNCTIONAL REQUIREMENTS

4.1 Upload Receipt Image

Table R4-1 Functional Requirements

FR1-1	Receipt Scanner shall support the following image formats ➤ JPG ➤ PNG ➤ HEIC
FR1-2	Receipt Scanner shall process each receipt individually.

4.2 Store Uploaded File

Table R4-2 Functional Requirements

FR2-1	Receipt Scanner shall store uploaded receipt images.
FR2-2	Receipt Scanner shall verify that uploaded files are valid image formats.

4.3 Perform OCR

Table R4-3 Functional Requirements

FR3-1	Receipt Scanner shall perform Optical Character Recognition on each uploaded receipt image.
-------	--

4.4 Extract Receipt Fields

Table R4- Functional Requirements

FR4-1	Receipt Scanner shall attempt to identify the fields from the OCR output: ➤ Vendor name ➤ Transaction date ➤ Total price paid ➤ Gallons purchased
--------------	--

4.5 Display Extracted Results

Table R5- Functional Requirements

FR5-1	Receipt Scanner shall display the extracted receipt information to the user.
FR5-2	Receipt Scanner shall display summary statistics for the receipt collection.

4.6 Allow Manual Correction

Table R4-6 Functional Requirements

FR6-1	Receipt Scanner shall allow the user to manually edit extracted receipt fields.
--------------	--

4.7 Save or Export Results

Table R4-7 Functional Requirements

FR7-1	Receipt Scanner shall retain receipt data within the application interface for the current session.
--------------	--

This Page Intentionally Left Blank

CHAPTER 5

INTERFACE REQUIREMENTS

5.1 User Interface

Table R5-1 Interface Requirements

IR1-1	Receipt Scanner shall provide a browser-based interface allowing users to: ➤ Upload receipt images ➤ View extracted receipt data ➤ Edit extracted fields ➤ View summary statistics
--------------	---

5.2 Software Interfaces

Table R5-2 Interface Requirements

IR2-1	Receipt Scanner shall interact with an OCR engine to convert receipt images into machine-readable text.
--------------	--

This Page Intentionally Left Blank

CHAPTER 6

NON-FUNCTIONAL REQUIREMENTS

6.1 Usability

Table R6-1 Non-Functional Requirements

NFR-1-1	Receipt Scanner shall provide a simple and intuitive web interface suitable for users with minimal technical experience.
----------------	---

6.2 Reliability

Table R6-2 Non-Functional Requirements

NFR-2-1	Receipt Scanner shall identify receipts that fail field extraction and flag them as problematic rather than using them in summary calculations.
----------------	--

6.3 Maintainability

Table R6-3 Non-Functional Requirements

NFR-3-1	Receipt Scanner shall be implemented in modular Python components to allow future improvements to OCR processing or field extraction logic.
----------------	--

This Page Intentionally Left Blank

CHAPTER 7

ACADEMIC REQUIREMENTS

7.1 Assignment Requirements

Table R7-1 Academic Requirements

AR-5-1	Receipt Scanner shall utilize Natural Language Processing, AI, or Intelligent Image Processing.
AR-5-2	Each Group Member shall participate in the coding process.

7.2 External Requirements

Table R7-1 Academic Requirements

AER-5-1	All external libraries utilized will be specifically called out and included on the presentation
AER-5-2	Documentation for the project shall include all aspects of planning, documentation, and usability.

This Page Intentionally Left Blank

SOFTWARE DESIGN

RECEIPT SCANNER VOL 3

This Page Intentionally Left Blank

CHAPTER 1

INTRODUCTION

1.1 Purpose

This Software Design Description (SWDD) defines the internal architecture and design of the Receipt Scanner system. The purpose of this document is to describe how the system implements the functional and non-functional requirements defined in the Software Requirements Specification.

1.2 Scope

This document describes the design of the Receipt Scanner software system, which is implemented using Python. The design focuses primarily on the internal processing engine responsible for receipt analysis and data extraction.

The system supports the following primary capabilities:

- ingestion of receipt images
- OCR extraction of text from images
- parsing of receipt fields
- validation and flagging of problematic receipts
- organization of receipts into collections
- generation of summary statistics for collections

External interfaces such as browser-based frontends are treated as adapters that invoke the core Python services. The SWDD therefore focuses on the design of the internal processing components rather than the implementation of any particular user interface.

This Page Intentionally Left Blank

CHAPTER 2

SYSTEM ARCHITECTURE

The receipt Scanner system design uses a modular architecture that allows the separation of the receipt's core functionality from the user interface. This allows the system to solely focus on the core problem which is extracting structured data from the receipt images while also being flexible in how the results are used.

The system architecture consists of the following components who will perform a specific task in the receipt processing pipeline:

- Image input Module
- OCR Processing Module
- Text Processing and Classification Module
- Data Structuring Module
- Interface Layer

The modular design ensure that the main processing engine will function independently from the interface used to display the results. For example, the structured data receipt can be presented via web interface, exported to a file for further analyst or presented via command line interface.

This Page Intentionally Left Blank

CHAPTER 3

SYSTEM DESIGN OVERVIEW

3.1 Design Objectives

The design of the Receipt Scanner system is guided by the following objectives:

3.1.1 Modularity

The system is structured into distinct functional components responsible for specific tasks such as OCR extraction, text parsing, and summary generation. While the implementation consolidates some orchestration logic for simplicity, the conceptual separation of responsibilities is maintained to support future refactoring into fully independent modules.

3.1.2 Interface Independence

The system is designed such that the core receipt processing logic is independent of the user interface. In the current implementation, the web interface directly invokes processing components. Future enhancements would introduce a dedicated service layer to fully decouple interface and processing logic in future iterations.

3.1.3 Maintainability

The system uses idiomatic Python programming practices and clear component boundaries to allow future improvements or modifications without requiring significant redesign.

3.1.4 Extensibility

The design allows core components such as the OCR processor or receipt parser to be replaced with alternate implementations without affecting other parts of the system.

3.1.5 Simplicity

As a proof-of-concept system, the design prioritizes simplicity and clarity over complex architectural patterns. This ensures ease of understanding, rapid prototyping, and effective demonstration of core functionality.

3.2 Architectural Overview

The Receipt Scanner system follows a simplified layered architecture adapted for a proof-of-concept implementation. The intended architecture consists of three primary layers:

3.2.1 Interface Layer

The interface layer provides a browser-based user interface that allows users to upload receipts, view extracted data, and interact with the system.

3.2.2 Application Service Layer

The application processing layer is responsible for orchestrating the receipt processing workflow, including OCR execution, text parsing, validation, and summary generation.

3.2.3 Infrastructure Layer

The infrastructure layer provides supporting functionality such as image handling, OCR processing, and local storage of uploaded files. This includes integration with external libraries such as Tesseract OCR and image processing utilities.

CHAPTER 4

COMPONENT DESIGN

The core system is composed of several primary software components. Each component is responsible for a specific function within the receipt processing pipeline.

4.1 Receipt Ingestion Component

The Receipt Ingestion Component is responsible for receiving receipt images from external interfaces and preparing them for processing.

Responsibilities include:

- accepting uploaded image files
- validating file format
- storing receipt images in the local filesystem
- initiating the receipt processing workflow

Supported input formats include:

- JPG
- PNG
- HEIC

If a file does not match the supported image formats, the ingestion component rejects the file before processing.

4.2 Image Preparation Component

The Image Preparation Component is responsible for preparing receipt images for OCR processing.

Responsibilities include:

- resizing and normalizing images
- converting images to appropriate formats for OCR
- enhancing contrast or clarity where necessary

This component improves OCR accuracy by ensuring that input images meet processing requirements.

4.3 OCR Processing Component

The OCR Processing Component extracts textual content from receipt images.

Responsibilities include:

- invoking the OCR engine (e.g., Tesseract)
- converting image data into raw text output
- handling OCR failures or disabled OCR configurations

The system allows OCR functionality to be optionally disabled for testing or demonstration purposes, in which case deterministic or placeholder data may be used.

4.4 Receipt Parsing Component

The Receipt Parsing Component processes OCR output to extract structured data fields.

Responsibilities include:

- identifying vendor name, date, total amount, and gallons purchased
- applying pattern matching (e.g., regular expressions)
- normalizing extracted values into consistent formats

This component represents the core logic of transforming unstructured OCR text into usable data.

To accomplish these objectives, regex will be utilized to search and find specific text structure.

4.5 Manual Correction Component

The Manual Correction Component allows users to modify extracted data when OCR or parsing errors occur.

Responsibilities include:

- enabling user edits to extracted fields
- updating stored receipt data
- ensuring corrected data is used in summary calculations

This improves overall system reliability despite imperfect OCR accuracy.

The need for manual correction will be any failure to extract a field with OCR.

4.6 Application Service Component

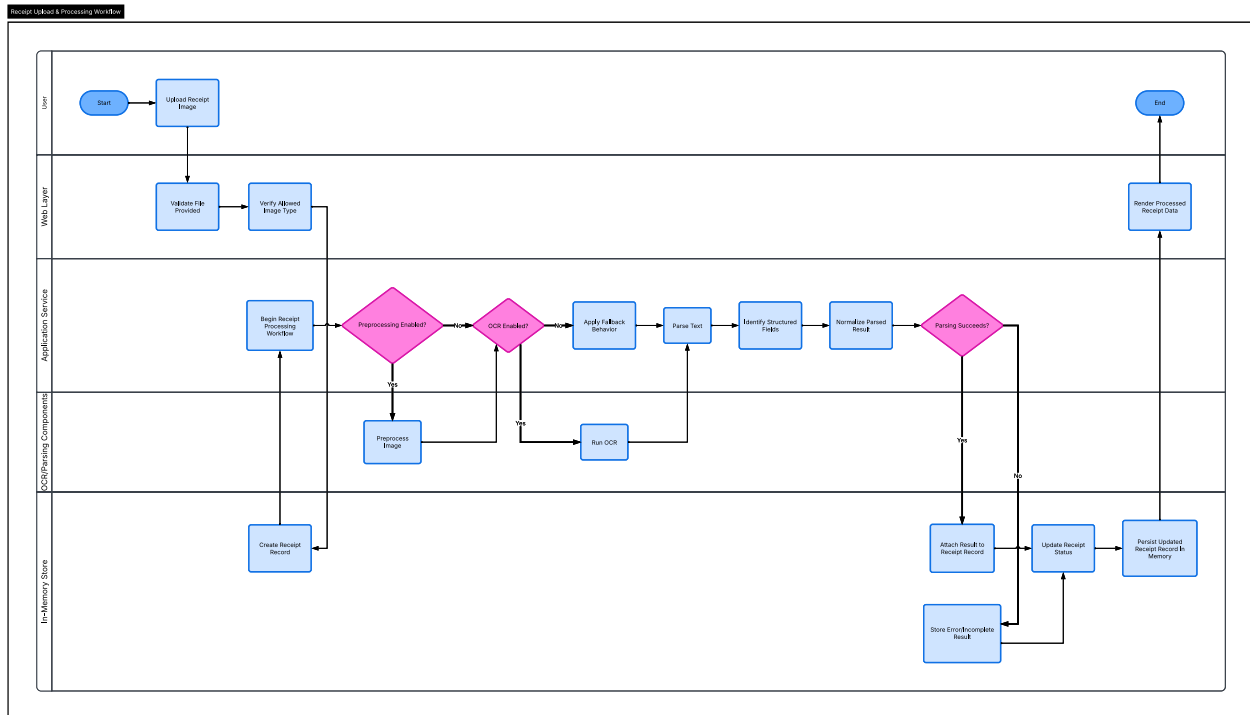
The Application Service Component coordinates the overall processing workflow.

Responsibilities include:

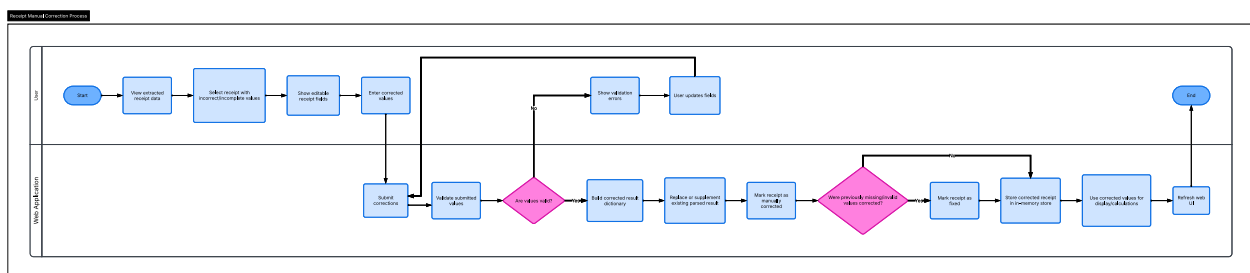
- managing receipt ingestion and processing sequence
- invoking OCR and parsing components
- handling data flow between components
- preparing results for presentation in the interface

CHAPTER 5 PROCESSING WORKFLOWS

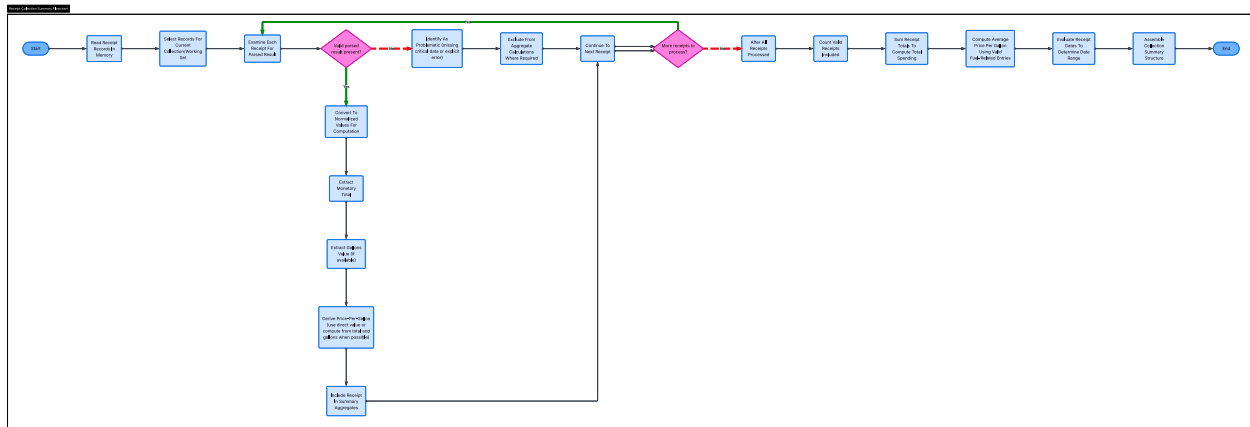
5.1 Single Receipt Processing Flow



5.2 Manual Correction Flow



5.3 Summary Generation Flow



This Page Intentionally Left Blank

CHAPTER 6

ERROR HANDLING AND CONSTRAINTS

6.1 Error Handling

The Receipt Scanner system incorporates basic error handling mechanisms appropriate for a proof-of-concept application. The goal of error handling within the system is to ensure that failures in OCR processing, data extraction, or user input do not cause system crashes and that users are provided with clear, actionable feedback.

6.1.1 Input Validation Errors

The system validates all uploaded files to ensure they are supported image formats (JPG, PNG, HEIC). If an invalid file type is uploaded, the system will display “error” in the status of the upload. The user can still adjust the failed upload fields (such as date, total, etc), but OCR will not be performed.

6.1.2 OCR Failures

OCR processing may fail due to missing dependencies (e.g., Tesseract not installed), poor image quality, or configuration settings. The system handles these scenarios by returning empty results when OCR fails

6.1.3 Data Extraction Errors

Because receipt formats vary widely, the parsing process may fail to correctly identify required fields such as vendor name, date, total, or gallons. In such cases:

- The system flags the receipt as incomplete
- Missing or invalid fields are set to empty values
- The receipt is be excluded from summary calculations

6.1.4 Summary Calculation Errors

Errors during aggregation (e.g., division by zero when calculating averages) are handled by:

- Skipping invalid or incomplete records
- Defaulting summary values to zero or null when insufficient data is available

Overall, error handling prioritizes system stability and user awareness while maintaining simplicity consistent with a proof-of-concept design.

If a manually entered field is invalid, 0 is the default.

6.2 Design Constraints

6.2.1 7.2.1 Proof-of-Concept Scope

The system is intentionally designed as a prototype to demonstrate OCR-based receipt processing. As a result:

- Advanced error recovery and robustness are limited
- The system is not intended for production or enterprise use
- Simplicity is prioritized over scalability and performance

6.2.2 OCR Limitations

OCR accuracy is inherently limited by:

- Image quality (blurriness, lighting, distortion)
- Variability in receipt formats
- Text alignment and formatting

As a result:

- Extraction accuracy cannot be guaranteed
- Manual correction is required for reliability

DESIGN VS IMPLEMENTATION CONSIDERATIONS

The current implementation of the Receipt Scanner system reflects a proof-of-concept realization of the proposed architecture. While the design specifies a clear separation between interface, service, and infrastructure layers, the implementation consolidates some of these responsibilities to simplify development and deployment.

Key deviations include:

- Partial integration of service-layer logic within the web application
- Use of lightweight in-memory data structures instead of formal data models
- Simplified module boundaries to prioritize rapid prototyping

Despite these simplifications, the system preserves the core architectural intent and demonstrates the feasibility of a modular receipt processing pipeline. The design supports future refactoring to fully align with the intended layered architecture.

This Page Intentionally Left Blank

USER GUIDE

RECEIPT SCANNER VOL 5

This Page Intentionally Left Blank

CHAPTER 1

INTRODUCTION

1.1 Purpose

The purpose of this User Guide is to provide clear and detailed instructions for installing, configuring, and using the Receipt Scanner application. This guide is intended to help users understand how to interact with the system, upload receipt images, view extracted data, and analyze receipt collections.

The User Guide is designed for users with minimal technical experience and provides step-by-step instructions to ensure that the application can be used effectively without requiring advanced knowledge of programming or software systems.

1.2 Intended Users

The Receipt Scanner application is designed for a wide range of users who need a simple and efficient way to digitize and analyze receipt data. Intended users include:

- Students working on academic projects involving data extraction and analysis
- Small business owners tracking expenses such as fuel purchases
- Individuals managing personal finances and receipts
- Users with limited technical background who require an easy-to-use interface

1.3 Overview of the System

The Receipt Scanner system allows users to upload images of receipts and automatically extract key information using Optical Character Recognition (OCR). The system processes the receipt images and identifies important fields such as vendor name, transaction date, total price, and gallons purchased.

The extracted data is then organized into a structured format and displayed to the user. Users can review and edit the extracted information if necessary and view summary statistics for their receipt collections.

This Page Intentionally Left Blank

CHAPTER 2

SYSTEM REQUIREMENTS

To properly run an instance of Receipt Scanner Locally, the following requirements must be met:

- Tesseract OCR engine installed

This Page Intentionally Left Blank

CHAPTER 3

INSTALLATION INSTRUCTIONS

To run Receipt Scanner locally, perform the following steps to install the software:

Step 1: Install Python

Download and install Python 3.x from the official Python website if it is not already installed on your system.

Step 2: Install Required Libraries

Open a terminal or command prompt and run the following command:

```
pip install -r requirements.txt
```

Step 3: Install Tesseract OCR

Download and install the Tesseract OCR engine for your operating system.

After installation, ensure that the Tesseract executable is added to your system's PATH environment variable so that it can be accessed by the application.

Step 4: Download Project Files

Download or clone the Receipt Scanner project files to your local machine.

Step 5: Verify Installation

Ensure that all dependencies are installed correctly by running the application and checking for errors.

This Page Intentionally Left Blank

CHAPTER 4

PROGRAM EXECUTION

4.1 Starting the Application

If accessing the application via the web, no installation is required to start. Simply navigate to <https://preview.ckplace.org>.

If accessing the application via local use, after performing the installation, launch the application via `python -m Marymount.edu.receiptscanner.web.py`.

4.2 Using the System

4.2.1 Uploading a Receipt Image

To upload a receipt image:

1. Click on the “Upload Receipt” button
2. Select an image file from your computer
3. Click “Submit”

The system will automatically process the receipt and extract text using OCR.

4.2.2 Viewing Receipt Data

After processing, the system will display the extracted receipt information. This typically includes:

- Vendor name
- Transaction date
- Total price
- Gallons purchased

The extracted data is presented in a structured format for easy reading.

4.2.3 Manual Edits

Users can manually edit the values.

To edit data:

1. Click on the field you wish to modify
2. Enter the correct value
3. Save the changes

This ensures that inaccurate OCR results can be corrected before analysis.

4.2.4 Managing Receipt Collections

Users can organize receipts into collections for easier management.

- Collections allow users to:
- Group related receipts
- View multiple receipts together
- Perform analysis on grouped data

4.2.5 Viewing Summary Statistics

The system automatically calculates summary statistics for receipt collections, including:

- Total amount spent
- Average price per gallon
- Number of receipts
- Date range of transactions

These statistics help users quickly analyze spending patterns.

4.3 Error Messages

The following error messages can appear in the program.

Table U4-3 Error Messages

ERROR MESSAGE	CAUSE	SOLUTION
OCR not working	Tesseract not installed	Install Tesseract OCR and verify PATH
Incorrect data extraction	Poor image quality	Upload a clearer image or edit manually
Application not starting	Missing dependencies	Reinstall required Python libraries
Image upload fails	Unsupported format	Use JPG, PNG, or HEIC format

4.4 Best Practices

To achieve the best results when using the Receipt Scanner system, users should follow these guidelines:

- Use clear, high-resolution images of receipts
- Ensure receipts are well-lit and not blurry
- Avoid folded or damaged receipts
- Review extracted data for accuracy

Following these practices will improve OCR accuracy and overall system performance.

This Page Intentionally Left Blank

APPENDIX A

This Page Intentionally Left Blank

NOMENCLATURE

Application Service Layer	The logical processing layer that coordinates OCR, parsing, and data handling.
Flask	A lightweight Python web framework used to implement the application interface.
Gallons Purchased	The quantity of fuel purchased, used for computing price per gallon.
Interface Layer	The component responsible for user interaction (web UI).
Infrastructure Layer	Supporting services such as OCR engines and file handling systems.
Manual Correction	The process by which users adjust incorrectly extracted data fields.
OCR (Optical Character Recognition)	A technology used to convert images of text into machine-readable text.
Parsed Data	Structured data extracted from OCR output, including fields such as vendor, date, and total.
Parsing	The process of analyzing OCR text to extract structured fields using rules or patterns.
Receipt Collection	A user-defined grouping of receipts used for aggregation and analysis.
Receipt Scanner	The software system developed in this project to process receipt images and extract structured data.

Regex (Regular Expression)	A pattern-matching technique used to identify structured data within text.
Session Storage	Temporary storage of data during runtime, not persisted after application termination.
Summary Statistics	Aggregated values calculated from a collection of receipts (e.g., total spend, average price).
Tesseract OCR	An open-source OCR engine used to extract text from receipt images.
Vendor	The merchant or business from which a purchase was made.

This Page Intentionally Left Blank

APPENDIX B

This Page Intentionally Left Blank

EXTERNAL LIBRARIES

argparse	Parses command-line arguments and enables optional CLI-based interaction with the application (e.g., running in different modes or configurations).
Flask	Provides the browser-based interface and handles HTTP requests, routing, and rendering.
json	Provides functionality for encoding and decoding data in JSON format for structured data handling and potential export/import operations.
pathlib	Provides cross-platform file path handling and file management.
Pillow	Used for image manipulation and preprocessing prior to OCR.
re	Text Parsing/Pattern Matching
sys	Provides access to system-level parameters and runtime environment variables, including command-line arguments and execution control.
Tesseract OCR	Extracts text from receipt images. Required for full functionality.
typing	Supports static type annotations in Python to improve code readability, maintainability, and developer tooling support.

uuid

Identifier Generation

This Page Intentionally Left Blank
